| | |
|---|---|
| 1/21/2025 | 9 AM - 10 AM |

**Database Security:**

Security and User Management

SQL Server has robust security features, including:

- Authentication: SQL Server supports both Windows authentication (uses Windows login credentials) and SQL Server authentication (uses SQL Server-specific login credentials).
- Permissions: SQL Server allows administrators to grant and revoke permissions to users and roles. Permissions define who can access what data and perform specific actions (SELECT, INSERT, DELETE, etc.).
- Roles: SQL Server has fixed server roles and database roles to manage user permissions.

## User, Role & Permission:

In SQL, **users**, **roles**, and **permissions** are critical components for managing access control and ensuring the security of the database. These concepts are used to define who can access the database, what they can do, and how their access is restricted.

To create a user in SQL Server, you can use the **CREATE USER** command. Users are associated with logins, which are used to authenticate the user into the SQL Server instance. A login is a server-level principal, while a user is a database-level principal. You typically create a login at the server level and then create a corresponding user at the database level to give access to specific databases.

Here's how to create a user in SQL Server, step-by-step:

**1. Create a Login**

A login allows the user to connect to the SQL Server instance.

**Syntax to create a login:**

sql

---

CREATE LOGIN [login_name] WITH PASSWORD = 'strong_password';

- **login_name**: The name of the login.
- **password**: The login's password. Ensure it's a strong password as per your server's security policies.

**Example:**

sql

---

CREATE LOGIN User1 WITH PASSWORD = 'SecureP@ssw0rd';

This creates a login named User1 with the password SecureP@ssw0rd.

**2. Create a User for a Database**

Once the login is created, you need to create a user for that login in a specific database, granting them access to it.

**Syntax to create a user:**

sql

---

USE [DatabaseName];  -- Switch to the specific database

CREATE USER [user_name] FOR LOGIN [login_name];

- **DatabaseName**: The name of the database where the user needs access.
- **user_name**: The name of the user within the database.

- **login_name**: The login that the user is associated with.

**Example:**

sql

---

USE MyDatabase;  -- Switch to the 'MyDatabase' database

CREATE USER User1 FOR LOGIN User1;

This creates a user named User1 in the MyDatabase database, linked to the User1 login.

**3. Assign Roles to the User (Optional)**

After creating the user, you may want to assign specific roles to the user to grant them certain permissions, such as read-only or read-write access.

**Syntax to assign a role:**

sql

---

ALTER ROLE [role_name] ADD MEMBER [user_name];

- **role_name**: The name of the role you want to assign (e.g., db_datareader, db_datawriter, db_owner).
- **user_name**: The name of the user you want to assign the role to.

**Example:**

Assign the db_datareader role to the user:

sql

---

ALTER ROLE db_datareader ADD MEMBER User1;

This grants the user User1 the permission to read data from all tables in the database.

**4. Grant Additional Permissions (Optional)**

If you need to assign more granular permissions to the user (e.g., SELECT, INSERT, UPDATE), you can use the **GRANT** command.

**Syntax to grant permissions:**

sql

---

GRANT [permission_type] ON [object] TO [user_name];

- **permission_type**: The permission you want to grant (e.g., SELECT, INSERT, UPDATE, DELETE).
- **object**: The object (e.g., a table or view) that the permission is applied to.
- **user_name**: The name of the user.

**Example:**

Grant the SELECT permission on the Employees table to User1:

sql

---

GRANT SELECT ON Employees TO User1;

**Full Example: Creating a Login, User, and Assigning Permissions**

Here's a full example that creates a login, a user in a database, assigns roles, and grants permissions:

sql

---

-- Step 1: Create the login

CREATE LOGIN User1 WITH PASSWORD = 'SecureP@ssw0rd';

-- Step 2: Create the user in the database

USE MyDatabase;  -- Switch to the 'MyDatabase' database

CREATE USER User1 FOR LOGIN User1;

-- Step 3: Assign roles to the user (optional)

ALTER ROLE db_datareader ADD MEMBER User1;  -- Read-only access to the database

-- Step 4: Grant additional permissions (optional)

GRANT SELECT, INSERT ON Employees TO User1;  -- Allow SELECT and INSERT on the 'Employees' table

**5. Verify User Creation**

To verify that the user has been created successfully, you can query the sys.database_principals view.

sql

---

SELECT name, type_desc FROM sys.database_principals WHERE type_desc = 'SQL_USER';

This will list all users of the current database.

**Conclusion**

- **Logins** are created at the server level to authenticate the user.
- **Users** are created at the database level to grant access to specific databases.
- You can assign roles (e.g., db_datareader, db_datawriter) to users to grant predefined permissions.
- You can also use the GRANT statement to provide more granular control over permissions, such as SELECT, INSERT, or UPDATE on specific tables.

**REVOKE THE PERMISSION**

In SQL Server, the **REVOKE** statement is used to remove previously granted permissions from a user or role. This command allows you to revoke the access rights (such as SELECT, INSERT, UPDATE, DELETE, etc.) that were granted using the **GRANT** command.

**Syntax for REVOKE Command**

sql

---

REVOKE [permission_type] ON [object] TO [user_or_role];

- **permission_type**: The specific permission type that you want to revoke (e.g., SELECT, INSERT, UPDATE, DELETE).
- **object**: The name of the object (e.g., table, view, or procedure) on which the permission is granted.

- **user_or_role**: The user or role from which the permission will be revoked.

**Example 1: Revoking Permissions on a Table**

If you want to revoke the SELECT permission on the Employees table from User1, you would execute the following command:

sql

---

REVOKE SELECT ON Employees TO User1;

This will remove the SELECT permission from User1 on the Employees table.

**Example 2: Revoking Multiple Permissions**

You can also revoke multiple permissions at once. For example, if you want to revoke both SELECT and INSERT permissions from User1 on the Employees table, you can do the following:

sql

---

REVOKE SELECT, INSERT ON Employees TO User1;

**Example 3: Revoking Permissions from a Role**

If a user has been granted permissions through a role, you can revoke those permissions from the role. For instance, if User1 is a member of the db_datareader role and you want to revoke SELECT permission on a table from the role, you can do:

sql

---

REVOKE SELECT ON Employees TO db_datareader;

This will revoke SELECT permission on the Employees table from all members of the db_datareader role.

**Example 4: Revoking All Permissions**

To revoke all permissions on a table from a user, you can use the ALL keyword. For example, to revoke all permissions on the Employees table from User1:

sql

---

REVOKE ALL ON Employees TO User1;

This removes all permissions (e.g., SELECT, INSERT, UPDATE, DELETE, etc.) that were granted to User1 on the Employees table.

**Example 5: Revoking a Permission from a Specific User**

To revoke a specific permission (such as INSERT) from a user on a specific table, you can run:

sql

---

REVOKE INSERT ON Employees TO User1;

**Verifying Permissions after REVOKE**

To verify that the permissions have been revoked, you can query the sys.database_permissions system view:

sql

---

SELECT *

FROM sys.database_permissions

WHERE grantee_principal_id = USER_ID('User1');

This will return a list of permissions granted to User1. After a REVOKE statement, the relevant permissions should no longer appear.

---

| | |
|---|---|
| 1/21/2025_D | 10 AM - 11 AM |

**2. Role in SQL**

A **role** in SQL is a collection of permissions. Rather than assigning permissions individually to users, roles allow database administrators to group permissions into roles and assign these roles to users. This simplifies management and improves security, as users inherit permissions through roles.

Roles are useful because they allow for **role-based access control (RBAC)**, which ensures that users are granted the minimum necessary permissions based on their job function.

**Creating a Role:**

In SQL Server, you can create a role using the CREATE ROLE statement:

sql

---

CREATE ROLE role_name;

In MySQL, roles are also supported in versions 8.0 and later:

sql

---

CREATE ROLE 'role_name';

**Assigning Permissions to a Role:**

Once a role is created, you can grant permissions to it, and these permissions will be inherited by any user who is assigned the role.

sql

---

GRANT SELECT, INSERT ON Employees TO role_name;

In SQL Server:

sql

---

GRANT SELECT, UPDATE ON Orders TO role_name;

**Assigning Roles to Users:**

After creating a role and granting permissions to that role, you can assign the role to users:

sql

---

EXEC sp_addrolemember 'role_name', 'username';

In MySQL:

sql

---

GRANT 'role_name' TO 'username'@'hostname';

**Types of Roles:**

- **Built-in Roles**: Most databases have predefined roles that grant common sets of permissions, such as db_owner (can manage everything) or db_datareader (can read all data).
- **Custom Roles**: These are roles created by the DBA to fit specific needs, like a role for managing orders or customers.

---

**3. Permissions in SQL**

Permissions in SQL are the rights to perform certain actions on database objects (e.g., tables, views, procedures). Permissions can be granted to users or roles for various actions, such as reading or modifying data.

**Types of Permissions:**

- **SELECT**: Permission to read data from a table.
- **INSERT**: Permission to insert new rows into a table.
- **UPDATE**: Permission to modify existing data in a table.
- **DELETE**: Permission to delete data from a table.
- **EXECUTE**: Permission to execute a stored procedure or function.
- **ALTER**: Permission to modify the structure of a database object (e.g., adding a column to a table).
- **DROP**: Permission to delete a table, view, or other database objects.

**Granting Permissions:**

Permissions are granted to users or roles using the GRANT statement.

- **Granting Permissions to a User**:

sql

---

GRANT SELECT, UPDATE ON Employees TO 'username'@'hostname';

- **Granting Permissions to a Role**:

sql

---

GRANT SELECT, INSERT ON Orders TO 'role_name';

- **Revoking Permissions**: If you need to remove permissions, use the REVOKE statement:

sql

---

REVOKE INSERT ON Employees FROM 'username'@'hostname';

In SQL Server:

sql

---

REVOKE SELECT, INSERT ON Employees FROM username;

---

**4. Permission Hierarchy and Inheritance**

- **Permissions Inheritance**: When you grant permissions to a role, all users who belong to that role inherit those permissions. This makes it easier to manage permissions at a group level rather than individually.
- **Granting Permissions on a Table**: You can grant permissions on specific database objects (e.g., tables, views, stored procedures). For example:

sql

---

GRANT SELECT ON Employees TO 'username'@'hostname';

This grants the SELECT permission on the Employees table to the specified user.

- **Revoking Permissions**: Revoking permissions can be done at the user level or the role level, which will affect all users assigned to the role.

sql

---

REVOKE SELECT, INSERT ON Orders FROM 'role_name';

---

**5. SQL Server Built-in Roles**

SQL Server comes with a set of built-in roles that cover common database administrative tasks.

- **db_owner**: Has full control over the database (can perform any action).
- **db_securityadmin**: Manages security-related tasks like creating and managing roles and permissions.
- **db_datareader**: Can read all data in the database.
- **db_datawriter**: Can modify all data in the database.
- **db_ddladmin**: Can modify database objects like tables and views.
- **db_backupoperator**: Can back up and restore the database.

---

| | |
|---|---|
| 1/21/2025_D | 11 AM - 12 PM |

**Backup Recovery**

**Very Good Video: https://youtu.be/OR44SZNqFpg**


**Other Links:**

**How to create a backup Maintenance Plan in SQL Server**

**https://youtu.be/oLnZttZWWtk**

**https://youtu.be/ZskdROb0EaQ?list=PL5C95D74E7E658DCC**

Backup and Restore

To ensure data is not lost, SQL Server provides backup and restore functionalities:

- Backup: SQL Server allows you to back up databases, **including full backups, differential backups, and transaction log backups.**

Example:

sql

BACKUP DATABASE MyDatabase TO DISK = 'C:\backups\MyDatabase.bak';

- Restore: You can restore a database from a backup if needed.

Example:

sql

---

RESTORE DATABASE MyDatabase FROM DISK = 'C:\backups\MyDatabase.bak';

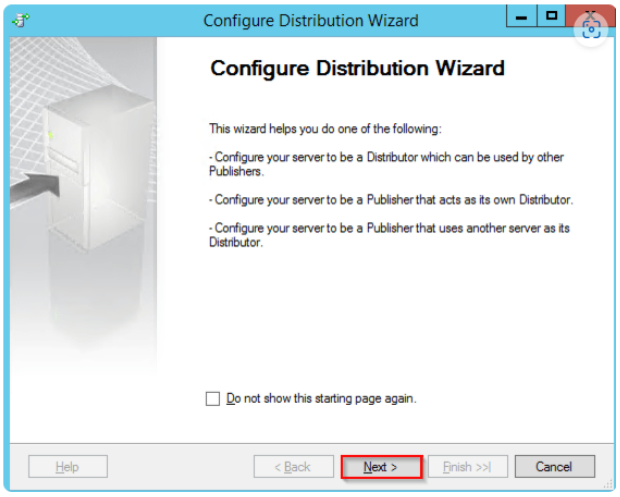
**Always on Availability:**

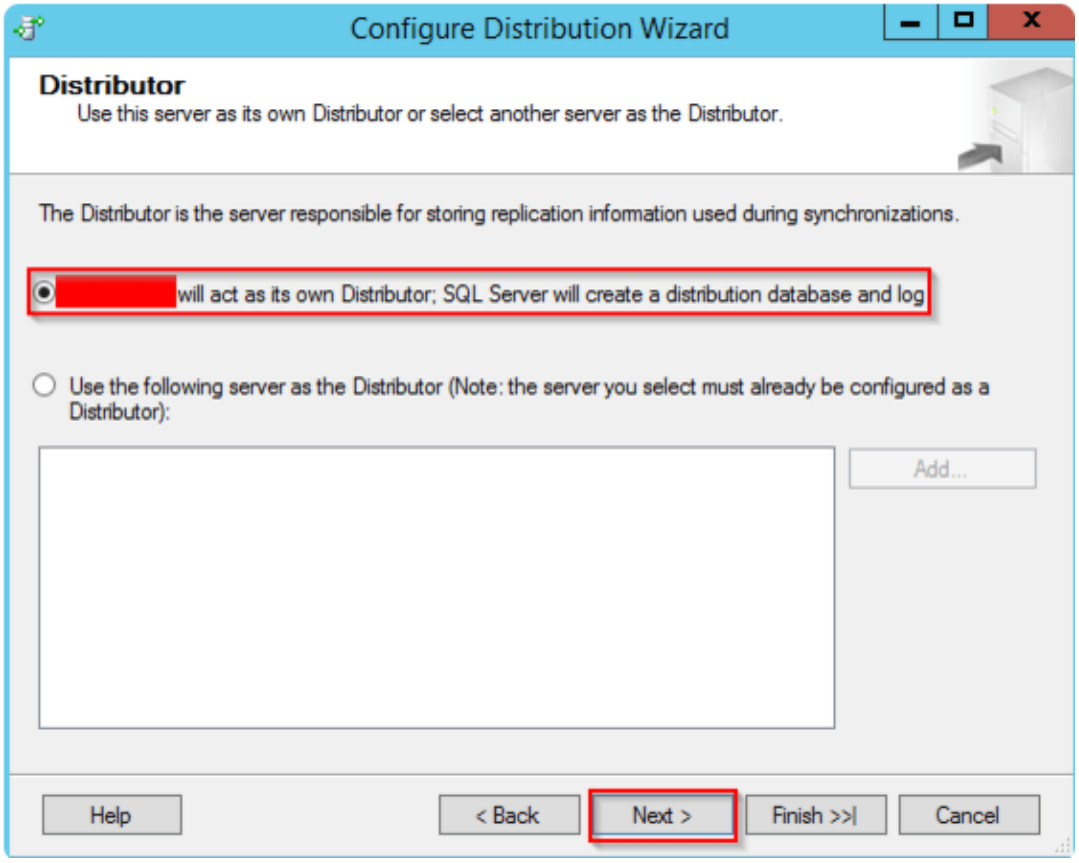
- https://youtu.be/3oOFoScCBTE

**Replication**

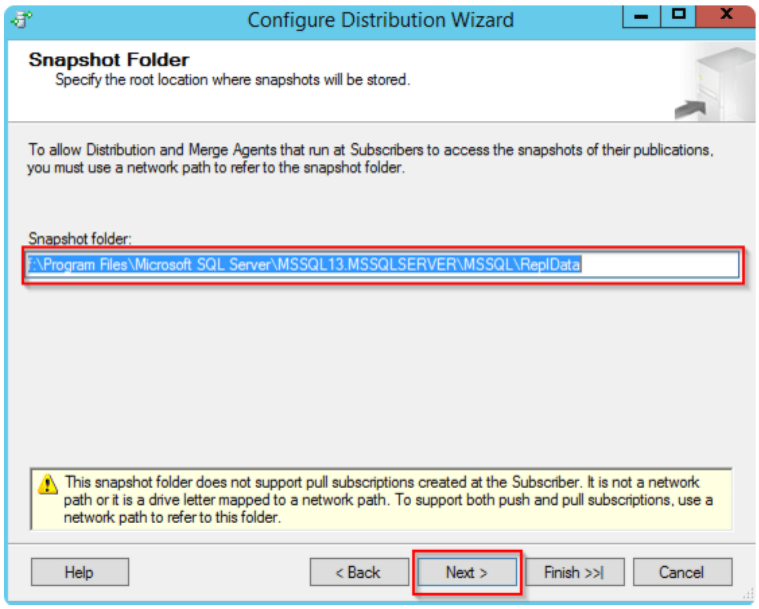
- **Step 1: Configuring the SQL Server Distributor**
- **Step 2: Configuring the SQL Server Publisher**
- **Step 3: Configuring the SQL Server Subscriber**

Create Distributor:

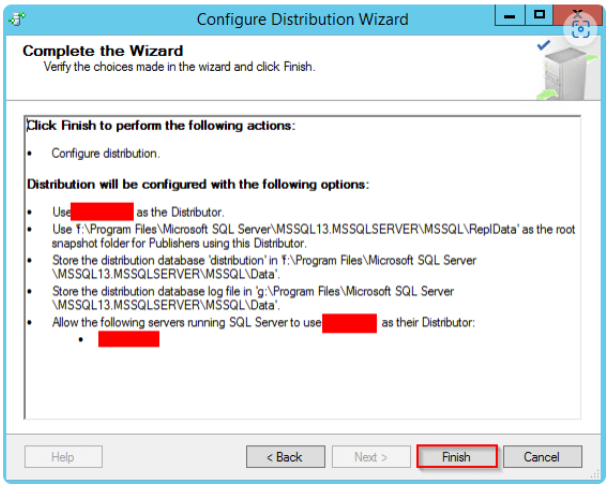



Select the network Path

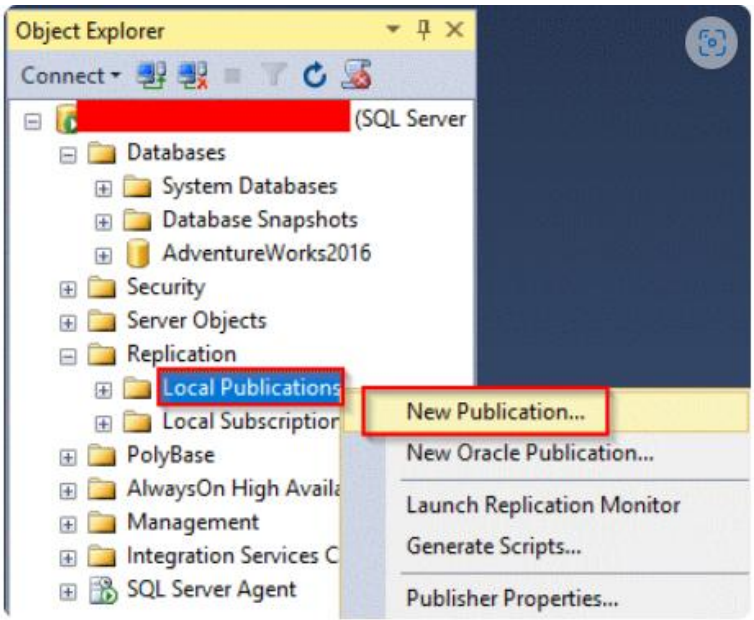

Configure Distribution Wizard
Snapshot Folder
Specify the root location where snapshots will be stored.
To allow Distribution and Merge Agents that run at Subscribers to access the snapshots of their publications, you must use a network path to refer to the snapshot folder.
Snapshot folder:
:\Program Files\Microsoft SQL Server\MSSQL13.MSSQLSERVER\MSSQL\ReplData
This snapshot folder does not support pull subscriptions created at the Subscriber. It is not a network path or it is a drive letter mapped to a network path. To support both push and pull subscriptions, use a network path to refer to this folder.
Help
< Back
Next >
Finish >>|
Cancel


Configure Distribution Wizard
Wizard Actions
Choose what happens when you click Finish.
At the end of the wizard:
Configure distribution
Generate a script file with steps to configure distribution
Help
< Back
Next >
Finish >>|
Cancel

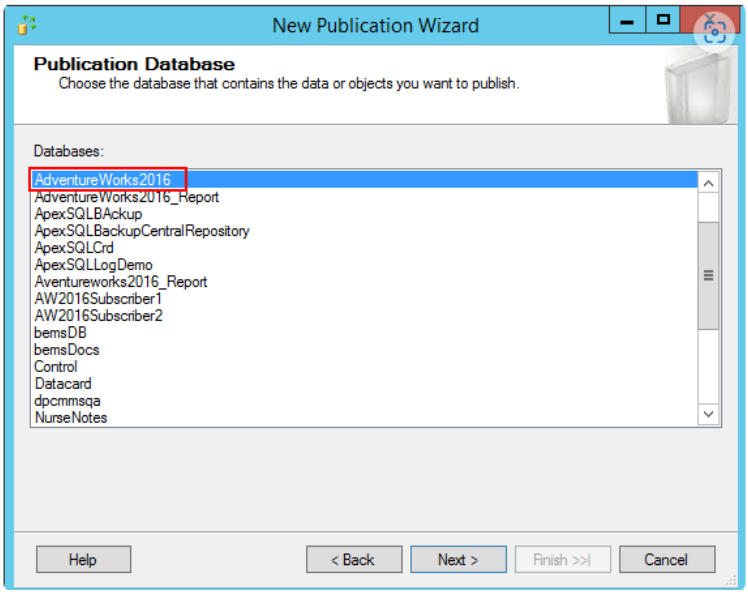


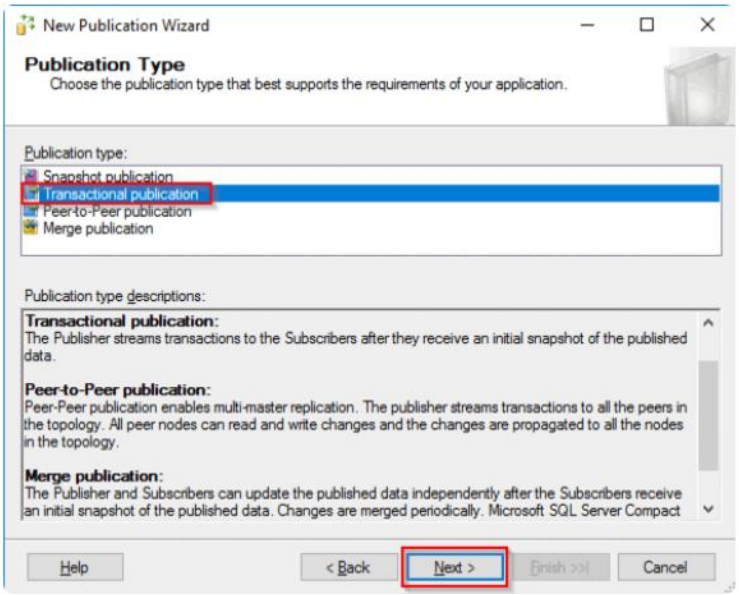

**Step 2: Configuring the SQL Server Publisher**

Expand the “Replication” folder from the “Object Explorer”. Right-click on “Local Publications” and choose “New Publication”.

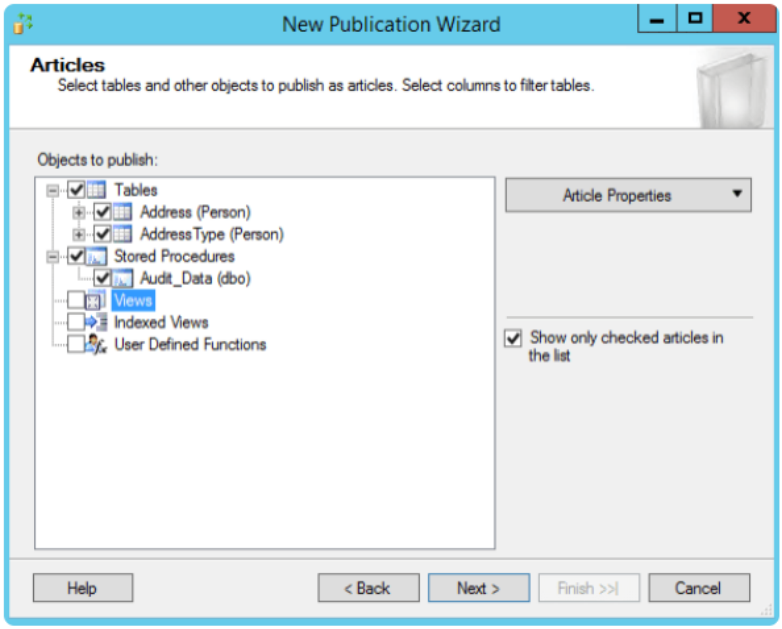


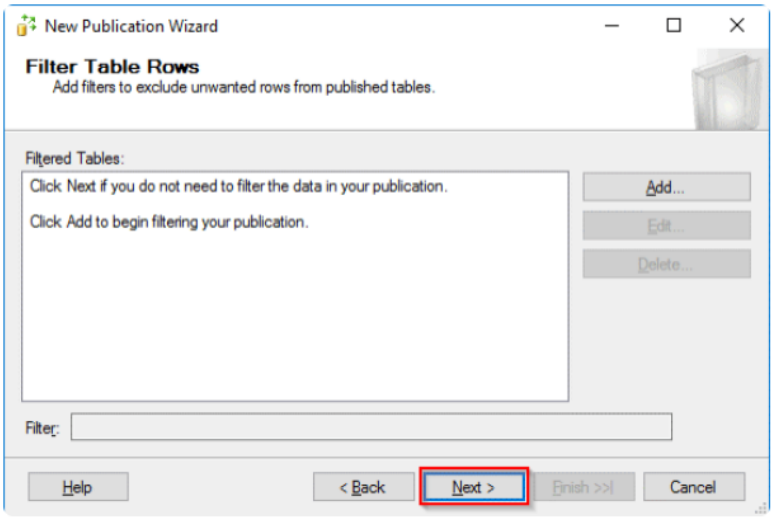

Choose the Database to replicate

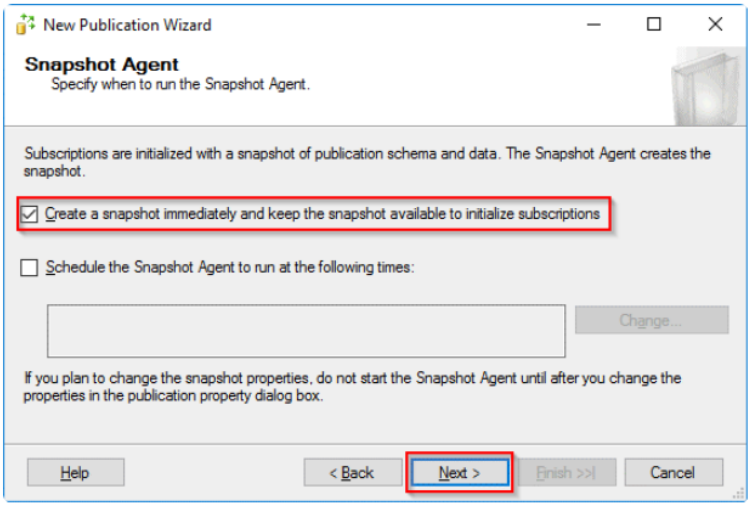


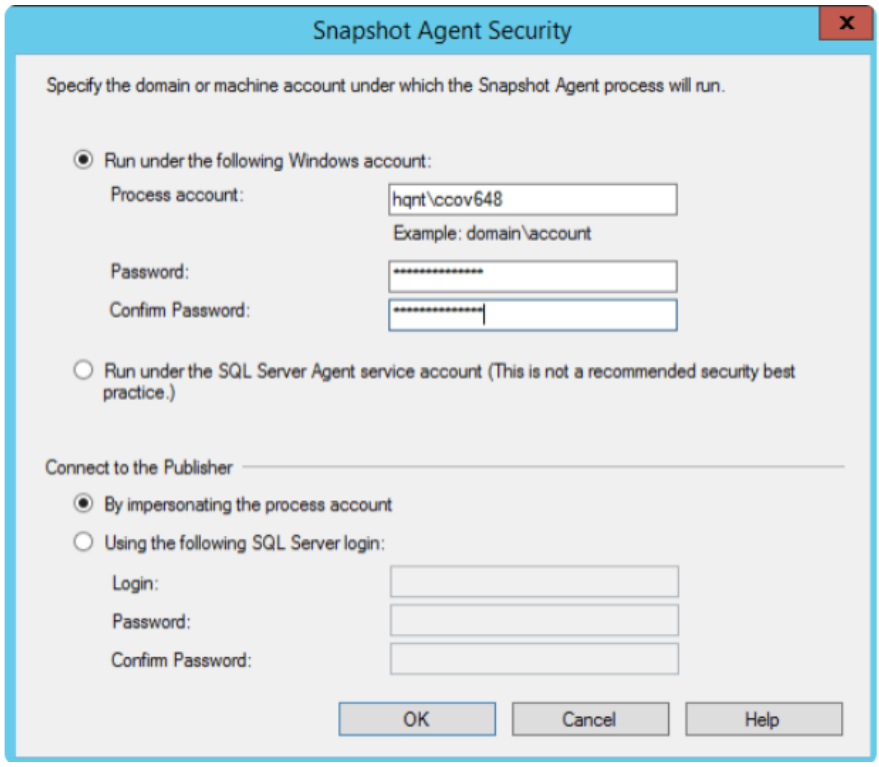



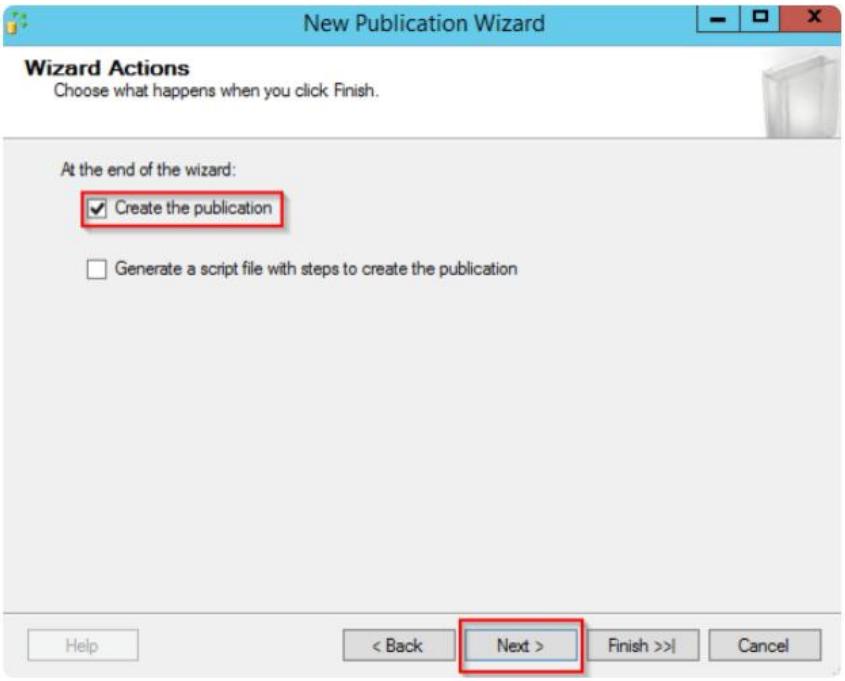
Choose Articles: (only primary key table will be shown)

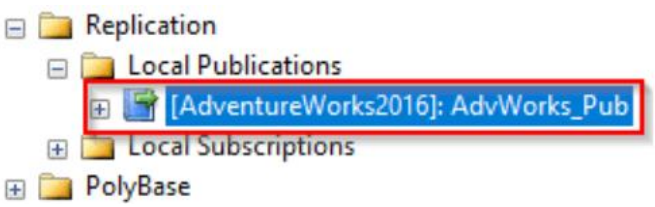


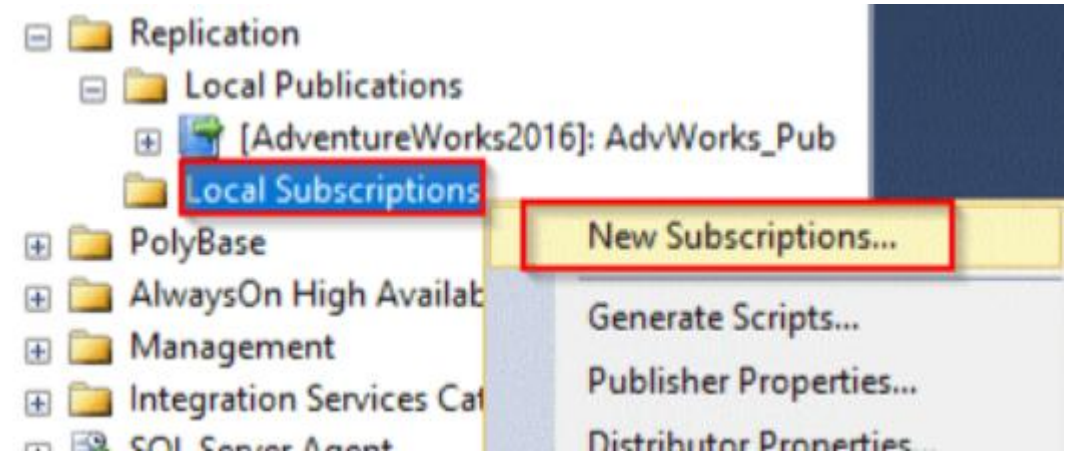



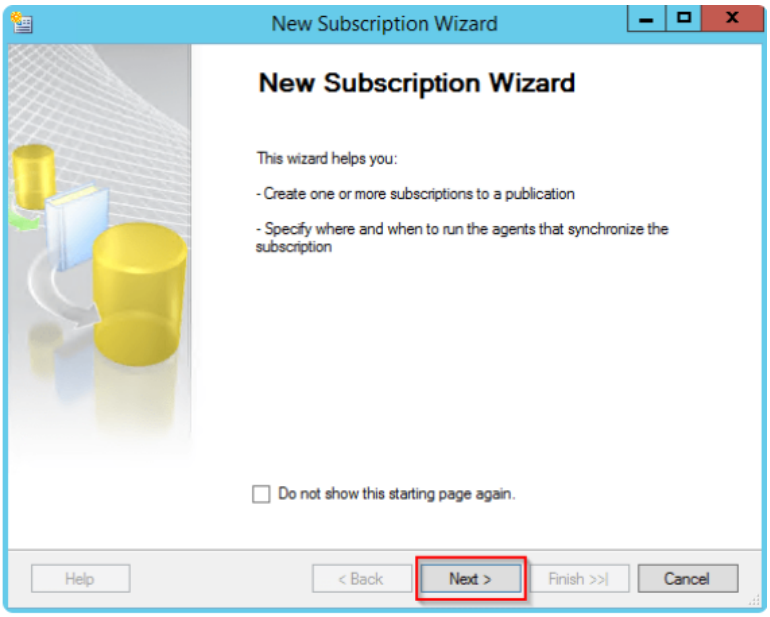

Snapshot Agent Security
Specify the domain or machine account under which the Snapshot Agent process will run.
Run under the following Windows account:
Process account:
hqnt\ccov648
Example: domain\account
Password:
Confirm Password:
Run under the SQL Server Agent service account (This is not a recommended security best practice.)
Connect to the Publisher
By impersonating the process account
Using the following SQL Server login:
Login:
Password:
Confirm Password:
OK
Cancel
Help

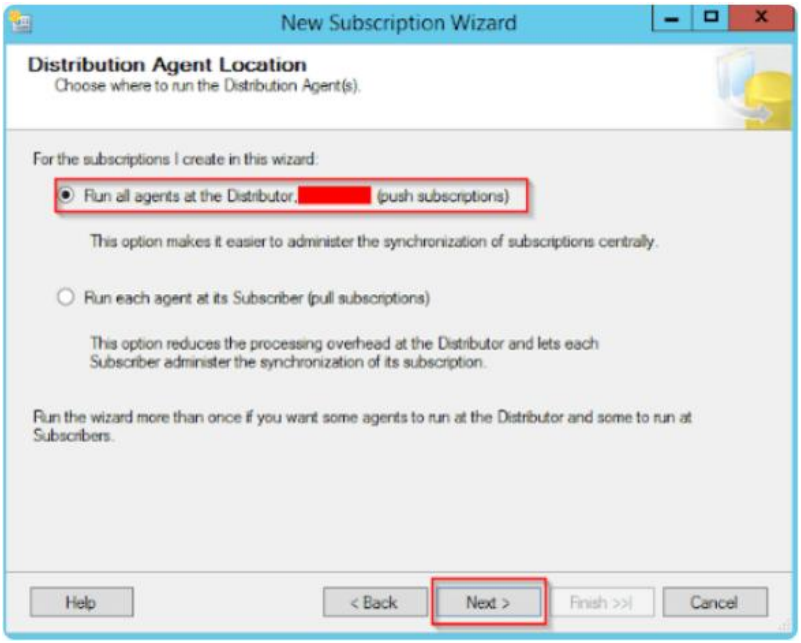

New Publication Wizard
Wizard Actions
Choose what happens when you click Finish.
At the end of the wizard:
Create the publication
Generate a script file with steps to create the publication
Help
< Back
Next >
Finish >>|
Cancel

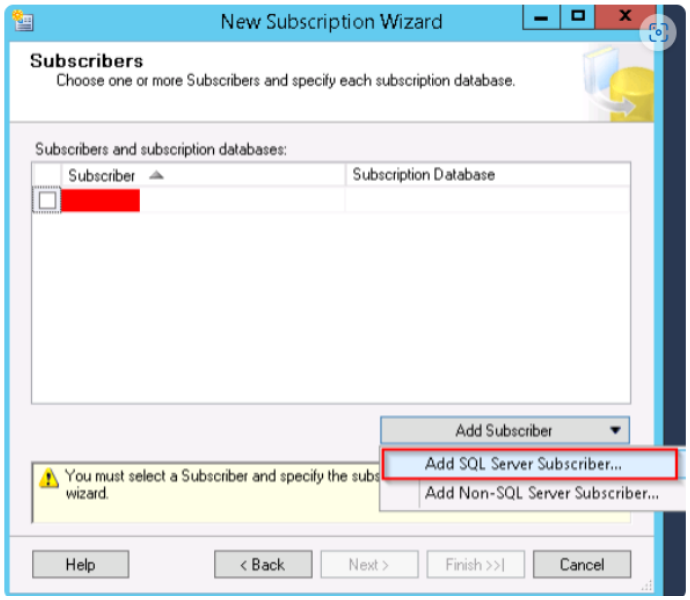

Replication
Local Publications
[AdventureWorks2016]: AdvWorks_Pub
Local Subscriptions
PolyBase

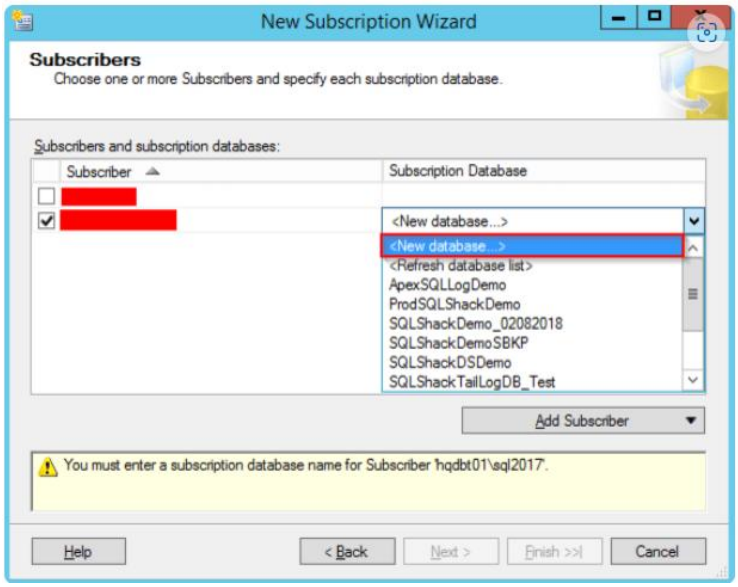
**Step 3: Configuring the SQL Server Subscriber**

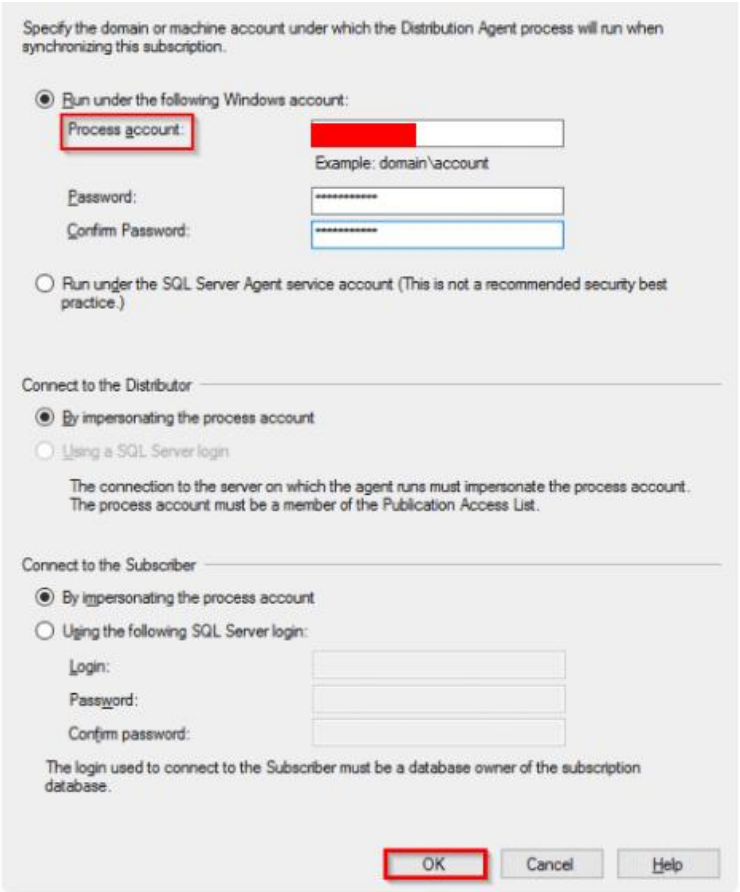


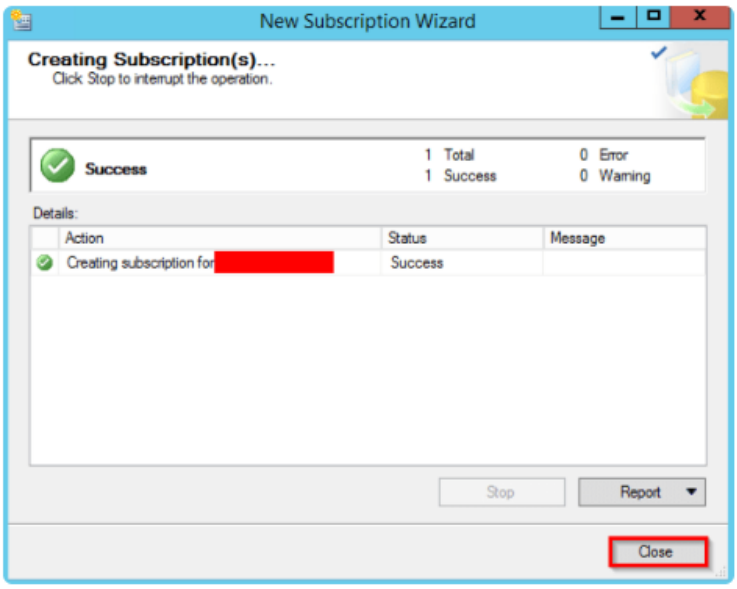



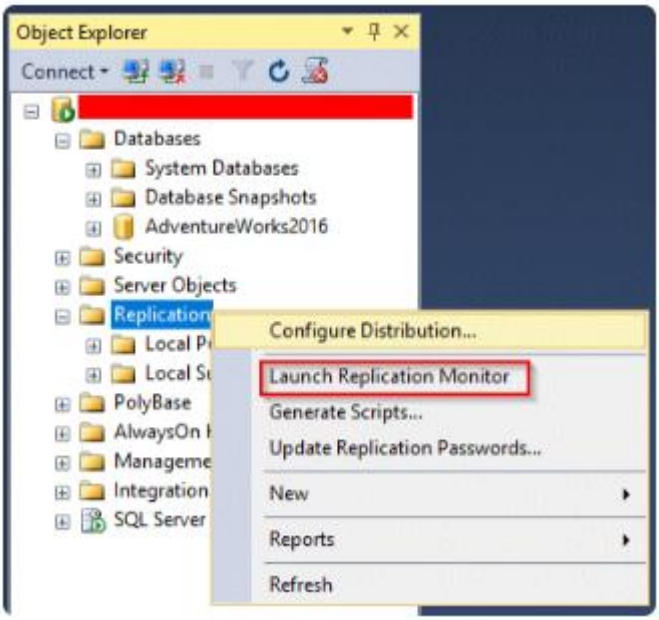

New Subscription Wizard
Distribution Agent Location
Choose where to run the Distribution Agent(s).
For the subscriptions I create in this wizard:
Run all agents at the Distributor, [redacted] (push subscriptions)
This option makes it easier to administer the synchronization of subscriptions centrally.
Run each agent at its Subscriber (pull subscriptions)
This option reduces the processing overhead at the Distributor and lets each Subscriber administer the synchronization of its subscription.
Run the wizard more than once if you want some agents to run at the Distributor and some to run at Subscribers.
Help
< Back
Next >
Finish >>|
Cancel


New Subscription Wizard
Subscribers
Choose one or more Subscribers and specify each subscription database.
Subscribers and subscription databases:
Subscriber
Subscription Database
Add Subscriber
Add SQL Server Subscriber...
Add Non-SQL Server Subscriber...
You must select a Subscriber and specify the subs
wizard.
Help
< Back
Next >
Finish >>|
Cancel


New Subscription Wizard
Subscribers
Choose one or more Subscribers and specify each subscription database.
Subscribers and subscription databases:
Subscriber
Subscription Database
<New database...>
<New database...>
<Refresh database list>
ApexSQLLogDemo
ProdSQLShackDemo
SQLShackDemo_02082018
SQLShackDemoSBKP
SQLShackDSDemo
SQLShackTailLogDB_Test
Add Subscriber
You must enter a subscription database name for Subscriber 'hqdbt01\sql2017'.
Help
< Back
Next >
Finish >>|
Cancel


Specify the domain or machine account under which the Distribution Agent process will run when synchronizing this subscription.
Run under the following Windows account:
Process account:
Example: domain\account
Password:
Confirm Password:
Run under the SQL Server Agent service account (This is not a recommended security best practice.)
Connect to the Distributor
By impersonating the process account
Using a SQL Server login
The connection to the server on which the agent runs must impersonate the process account. The process account must be a member of the Publication Access List.
Connect to the Subscriber
By impersonating the process account
Using the following SQL Server login:
Login:
Password:
Confirm password:
The login used to connect to the Subscriber must be a database owner of the subscription database.
OK
Cancel
Help




| | |
|---|---|
| 1/21/2025_D | 12PM - 1 PM |

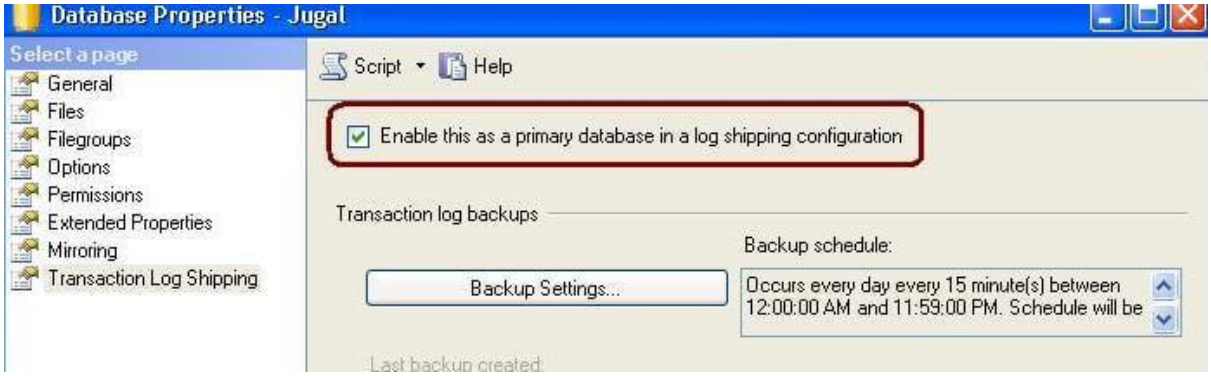
**Log Shipping:**

SELECT name, recovery_model_desc FROM sys.databases WHERE name = ‘Test’

```
USE [master]

GO

ALTER DATABASE [jugal] SET RECOVERY FULL WITH NO_WAIT

GO
```

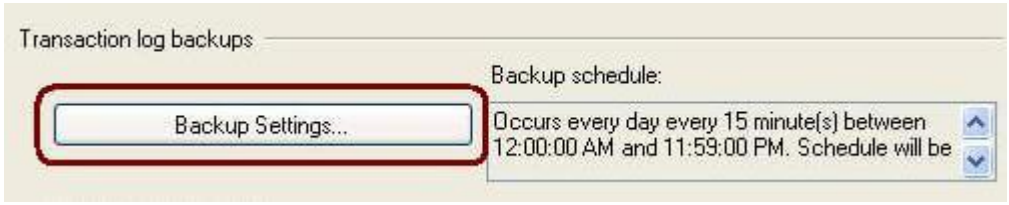
**Step 2**

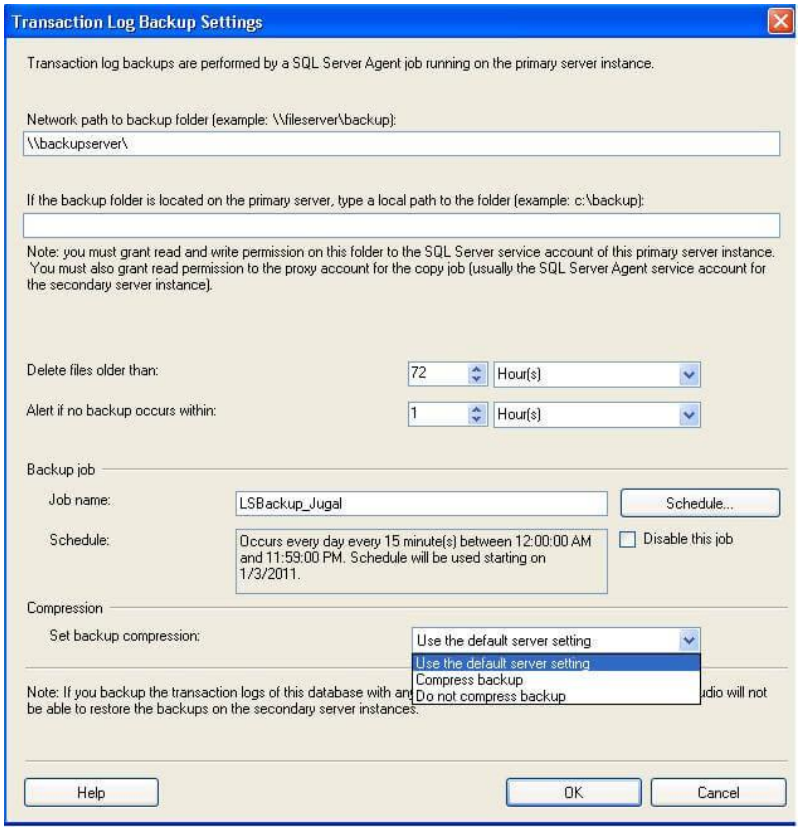
On the primary server, right click on the database in SSMS and select Properties. Then select the **Transaction Log Shipping** Page. Check the **"Enable this as primary database in a log shipping configuration"** check box.

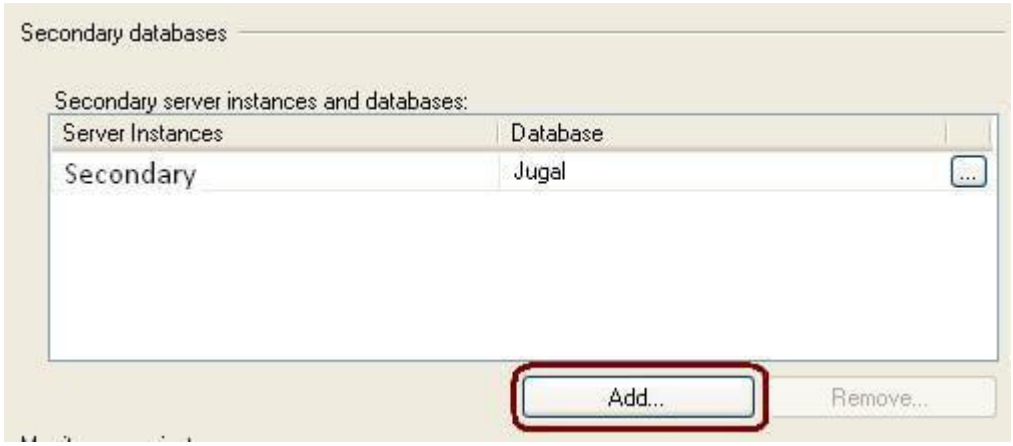



**Step 3**

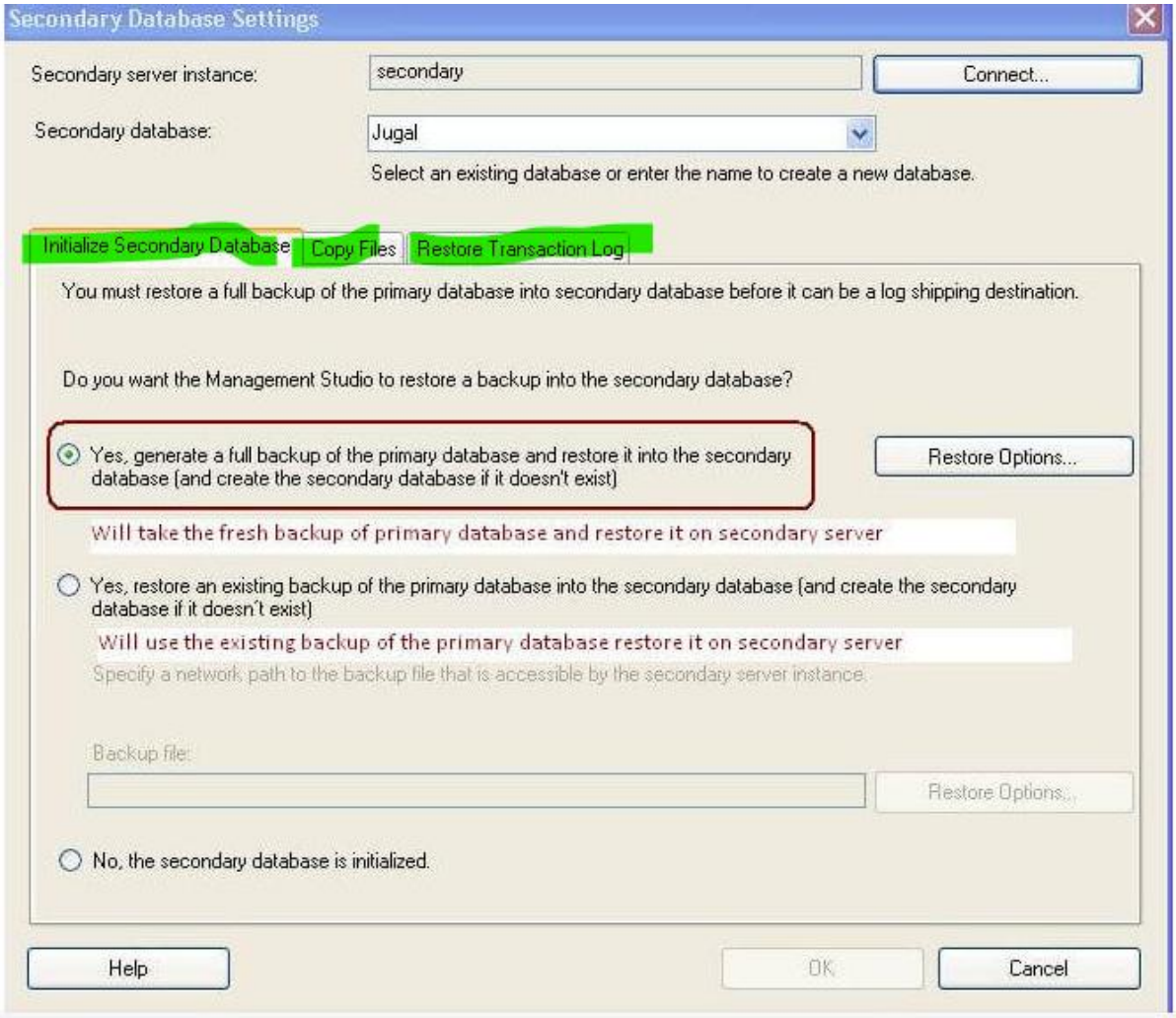
The next step is to configure and schedule a transaction log backup. Click on **Backup Settings...** to do this.

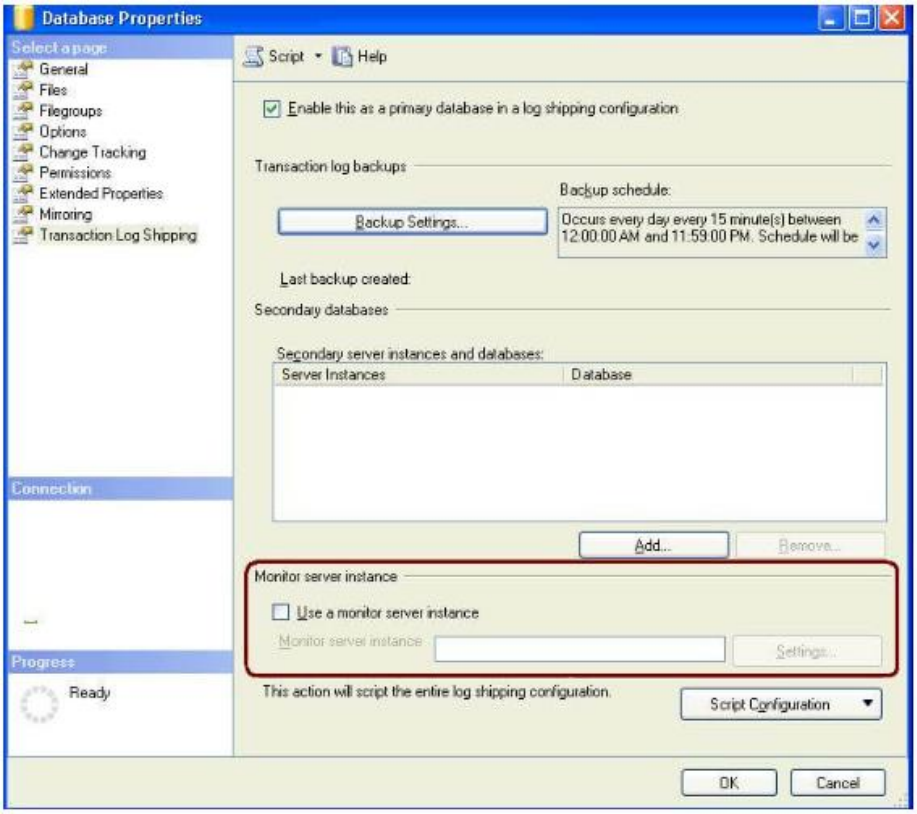


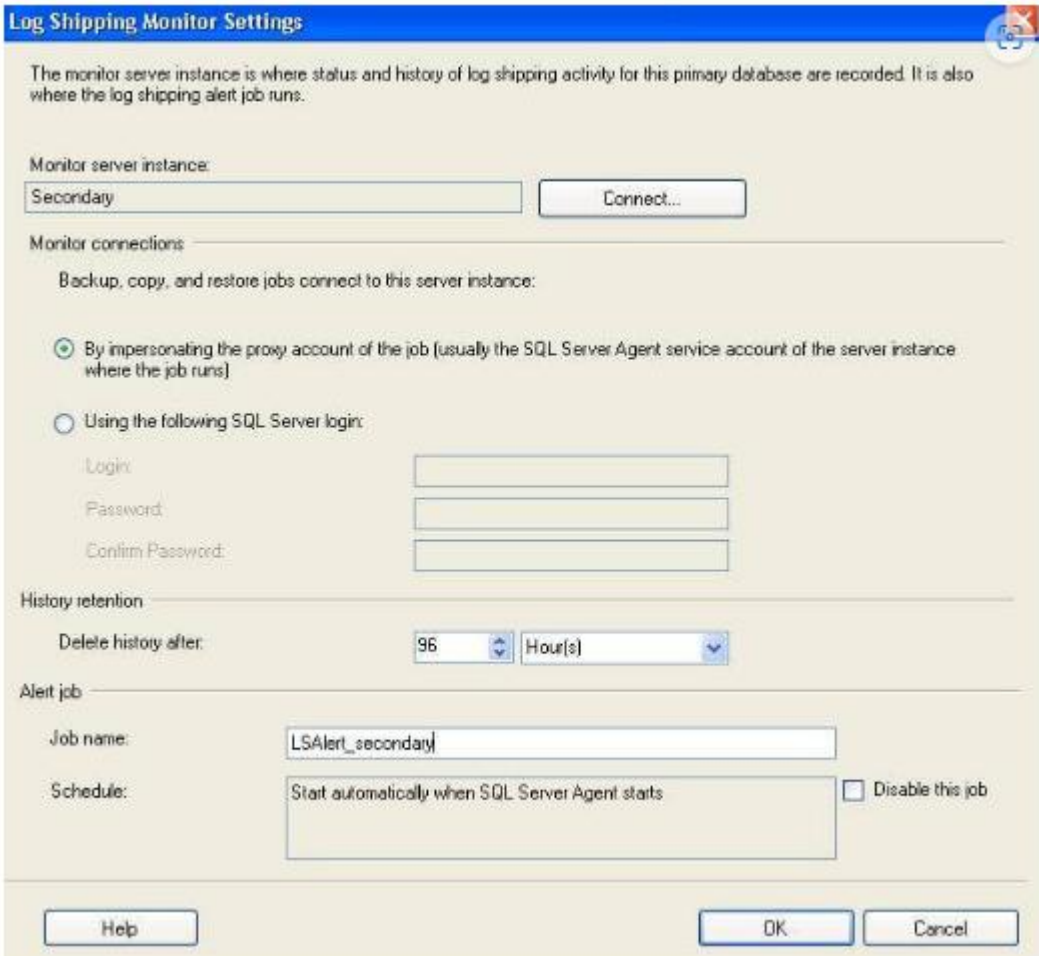


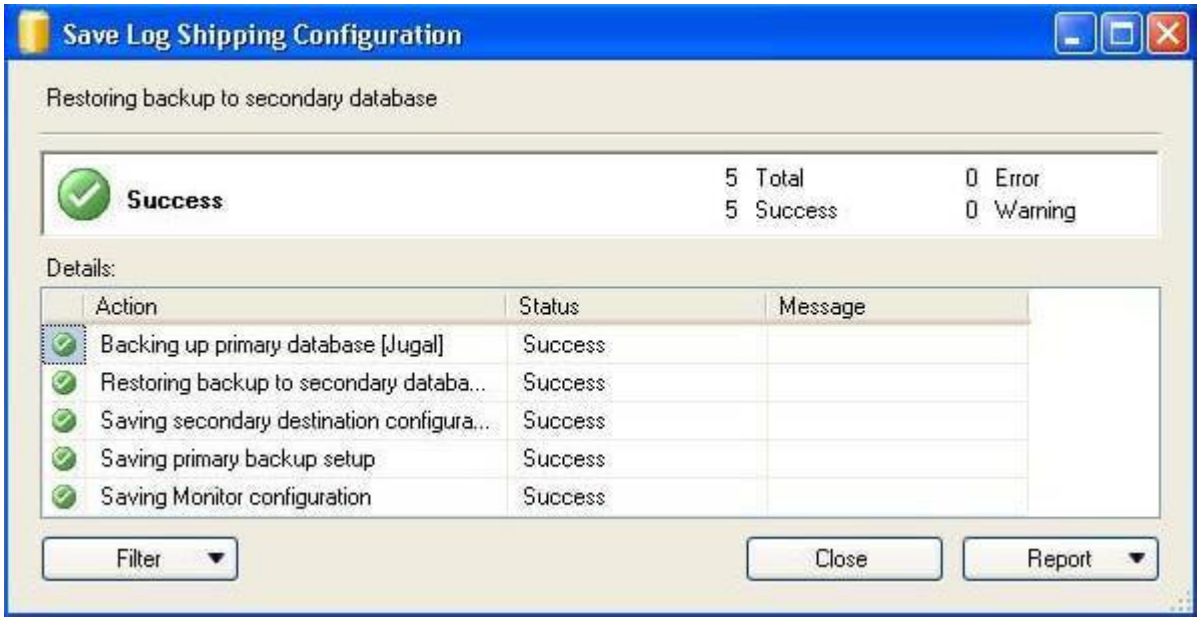




Configure: Initialize, Copy File and Restore


Secondary Database Settings
Secondary server instance:
secondary
Connect...
Secondary database:
Jugal
Select an existing database or enter the name to create a new database.
Initialize Secondary Database
Copy Files
Restore Transaction Log
You must restore a full backup of the primary database into secondary database before it can be a log shipping destination.
Do you want the Management Studio to restore a backup into the secondary database?
Yes, generate a full backup of the primary database and restore it into the secondary database (and create the secondary database if it doesn't exist)
Restore Options...
Will take the fresh backup of primary database and restore it on secondary server
Yes, restore an existing backup of the primary database into the secondary database (and create the secondary database if it doesn't exist)
Will use the existing backup of the primary database restore it on secondary server
Specify a network path to the backup file that is accessible by the secondary server instance.
Backup file:
Restore Options...
No, the secondary database is initialized.
Help
OK
Cancel


Database Properties
Select a page
General
Files
Filegroups
Options
Change Tracking
Permissions
Extended Properties
Mirroring
Transaction Log Shipping
Connection
Progress
Ready
Script
Help
Enable this as a primary database in a log shipping configuration
Transaction log backups
Backup schedule:
Backup Settings...
Occurs every day every 15 minute(s) between 12:00:00 AM and 11:59:00 PM. Schedule will be
Last backup created:
Secondary databases
Secondary server instances and databases:
Server Instances
Database
Add...
Remove...
Monitor server instance
Use a monitor server instance
Monitor server instance
Settings...
This action will script the entire log shipping configuration.
Script Configuration
OK
Cancel


Log Shipping Monitor Settings
The monitor server instance is where status and history of log shipping activity for this primary database are recorded. It is also where the log shipping alert job runs.
Monitor server instance:
Secondary
Connect...
Monitor connections
Backup, copy, and restore jobs connect to this server instance:
By impersonating the proxy account of the job (usually the SQL Server Agent service account of the server instance where the job runs)
Using the following SQL Server login:
Login:
Password:
Confirm Password:
History retention
Delete history after:
96
Hour(s)
Alert job
Job name:
LSAlert_secondary
Schedule:
Start automatically when SQL Server Agent starts
Disable this job
Help
OK
Cancel


Save Log Shipping Configuration
Restoring backup to secondary database
Success
5 Total
5 Success
0 Error
0 Warning
Details:
Action
Status
Message
Backing up primary database [Jugal]
Success
Restoring backup to secondary databa...
Success
Saving secondary destination configura...
Success
Saving primary backup setup
Success
Saving Monitor configuration
Success
Filter
Close
Report

| | |
|---|---|
| 1/21/2025_D | 2 PM - 3 PM |

**Practice Session**

| | |
|---|---|
| 1/21/2025_D | 3 PM - 4 PM |

**Stored Procedures and Functions and View:**

Stored procedures and functions are sets of SQL statements that can be stored in the database and executed when needed. A stored procedure performs actions (e.g., INSERT, UPDATE), while a function returns a value.

sql

---

```
CREATE FUNCTION function_name (parameter1, parameter2)
RETURNS data_type
BEGIN
  SQL statements;
  RETURN result;
END;
```

| | |
|---|---|
| 1/21/2025_D | 4 PM - 5 PM |

**Stored Procedure Syntax**:

CREATE PROCEDURE procedure_name (parameter1, parameter2)

BEGIN

SQL statements;

END;

- **Example**:

CREATE PROCEDURE GetEmployeeDetails(IN emp_id INT)

BEGIN

SELECT * FROM Employees WHERE EmployeeID = emp_id;

END;

**9. Query Optimization:**

SQL query optimization involves improving the performance of SQL queries by restructuring them, using indexes, and understanding how the database engine processes queries.

- **Best Practices**:
    - Use indexes wisely to speed up SELECT queries.
    - Avoid using SELECT *, as it may retrieve unnecessary columns.
    - Break complex queries into smaller ones, using temporary tables or CTEs.
    - Use EXPLAIN to analyze and understand the execution plan of a query.
- **Example**:

sql

---

EXPLAIN SELECT * FROM Employees WHERE Department = 'HR';

| | | |
|---|---|---|
| 1/21/2025_D | 5 PM - 6 PM | Practice Session |
| 1/21/2025_D | 6 PM - 7 PM | Practice Session |